ChaosFlow: Implementation Plan

Resilient Big Data Pipeline Platform

A configuration-driven Big Data pipeline builder with automated reliability and fault-injection testing

Graduation Project: Implementation Plan

Table of Contents

1. Introduction 4

2. Problem Statement 4

2.1 Repetitive development of Big Data pipelines . . . 4

2.2 Unverified reliability of distributed pipelines . . . 4

2.3 Research questions . . . 4

3. Aim and Objectives 4

3.1 Aim . . . 4

3.2 Objectives . . . 5

4. Scope 5

4.1 In scope . . . 5

4.2 Out of scope and assumptions . . . 5

5. Proposed System Overview 6

5.1 Layered architecture . . . 6

5.2 Data path . . . 6

5.3 Custom contribution versus reused infrastructure . . . 6

6. Technology Stack and Justification 7

7. Development Methodology 7

7.1 Iterative and incremental development with verification gates . . . 7

7.2 Phase gate (definition of done) . . . 7

7.3 Engineering practices . . . 7

7.4 Design principles and patterns . . . 8

8. Work Breakdown and Implementation Phases 8

Phase 1: Foundation . . . 8

Phase 2: Configuration Engine . . . 8

Phase 3: Plugin Architecture and Pipeline Execution Manager . . . 8

Phase 4: Kafka Integration . . . 9

Phase 5: Spark Integration . . . 9

Phase 6: HDFS Integration . . . 9

Phase 7: Airflow Integration . . . 9

Phase 8: Reliability Engine . . . 10

Phase 9: Metrics and Monitoring . . . 10

Phase 10: Authentication and Authorization . . . 10

Phase 11: Web Dashboard . . . 10

Phase 12: Test Consolidation and Continuous Integration . . . 11

Phase 13: AI Module . . . 11
Phase 14: Documentation and Final Integration . . . 11
9. Reliability Experiment Methodology 12
9.1 Experimental design . . . 12
9.2 Experiment lifecycle . . . 12
9.3 Fault catalog . . . 12
9.4 Metric definitions . . . 12
9.5 Data consistency validation . . . 13
9.6 Repeatability and result interpretation . . . 13
9.7 Safety protocol . . . 13
10. Verification and Validation 13
11. Evaluation Plan 13
12. Risk Analysis and Mitigation 14
13. Team Organization 14
14. Quality Assurance and Standards 15
15. Ethical, Legal, and Safety Considerations 15
16. Project Deliverables 16
17. Acceptance Criteria 16
18. Limitations and Future Work 16
18.1 Limitations . . . 16
18.2 Future work . . . 16
19. Conclusion 17
20. Selected References 17

1. Introduction

Modern organizations rely on distributed data pipelines to collect, clean, process, and store very large volumes of data. These pipelines are typically assembled from several independent technologies, such as a messaging system, a distributed processing engine, and a distributed file system. Building such pipelines is time-consuming, and verifying that they behave correctly when components fail is difficult and rarely done systematically before deployment.

This document presents the plan by which the project team will design, implement, verify, and evaluate **ChaosFlow**, a platform composed of two cooperating parts:

- **PipeForge**, which allows a user to describe a Big Data pipeline in a declarative YAML configuration and have the platform validate, generate, deploy, run, and monitor it automatically.
- **ChaosLab**, which allows a user to inject controlled, reversible failures into the running pipeline infrastructure and to measure, analyze, and report how the pipeline reacts.

The document describes the problem, objectives, scope, methodology, work breakdown, experimental design, verification strategy, evaluation criteria, risks, team organization, and deliverables of the project.

2. Problem Statement

2.1 Repetitive development of Big Data pipelines

Many Big Data applications share the same architecture: data is read from a source, published to a message broker, processed by a distributed engine, and written to distributed storage. Across projects, the infrastructure remains largely the same, while only the data source, schema, processing rules, and storage destination change. Developers nevertheless reconfigure and rewrite substantial parts of the pipeline for each new use case, which is slow and error-prone.

2.2 Unverified reliability of distributed pipelines

Distributed systems are subject to partial failures: a broker may become unavailable, a worker may crash, a network link may be interrupted, or a node may run out of CPU, memory, or disk. Teams often discover how their pipeline behaves under such conditions only during real incidents. There is a need for a systematic, automated, and repeatable way to apply controlled failures and to quantify the effects on recovery time, data integrity, throughput, and availability.

2.3 Research questions

- **RQ1.** Can a declarative configuration model capture enough of the structure of common Big Data pipelines to generate, deploy, and run them for different datasets without modifying the core engine?
- **RQ2.** Can controlled, reversible fault injection combined with automated measurement produce repeatable, quantitative reliability assessments of such pipelines?

3. Aim and Objectives

3.1 Aim

To design and implement a modular, extensible, and observable platform that generates Big Data pipelines from configuration and evaluates their resilience through automated, safe fault-injection experiments.

3.2 Objectives

- **O1.** Define a YAML-based pipeline specification and a validation engine that reports precise, field-level errors.
- **O2.** Implement a plugin architecture so that data sources, processing operations, and storage backends can be added without changing the core.
- **O3.** Implement a pipeline generator and execution manager that deploy and run pipelines over Kafka, Spark, and HDFS.
- **O4.** Implement a reliability engine supporting multiple fault types with strict safety controls, a defined experiment lifecycle, and automatic cleanup.
- **O5.** Define and compute reliability metrics (recovery time, data loss, throughput, availability, error rate, resource usage) and a transparent report for every experiment.
- **O6.** Provide monitoring, structured logging, a REST API, authentication with role-based access, and a web dashboard.
- **O7.** Provide an optional AI module for anomaly detection, risk indication, and explainable recommendations based on historical experiment data.
- **O8.** Provide automated tests, a continuous integration workflow, and documentation sufficient for another developer to reproduce the system.

4. Scope

4.1 In scope

- A local, containerized environment based on Docker Compose.
- Three data source types (CSV, JSON, REST API), eight processing operations, and storage in HDFS with additional storage plugins as extensions.
- Real Kafka, Spark, and HDFS components; no simulated substitutes.
- Seven fault types: container stop, container restart, network disruption, CPU stress, memory stress, disk pressure, and service delay.
- Metrics collection, Prometheus and Grafana dashboards, a reliability report generator, and historical storage of results.
- A web dashboard and a documented REST API.

4.2 Out of scope and assumptions

- Production or cloud deployment, and automatic deployment of any kind.
- Multi-machine clusters; the evaluation is performed on a single development machine, and its consequences for realism are documented.
- True hardware or data-center failures; faults are applied at the container level.
- Production-grade security hardening beyond reasonable development-level controls.
- The AI module is supplementary; the platform must be fully functional without it.

5. Proposed System Overview

5.1 Layered architecture

```
+-------------------------------------------------------------+
| Presentation   React + TypeScript dashboard                 |
+-------------------------------------------------------------+
| API            FastAPI REST API, JWT authentication, RBAC   |
+-------------------------------------------------------------+
| Application    Services: pipelines, experiments, reports    |
+-------------------------------------------------------------+
| Domain/Core    Pipeline model, validators, plugins,         |
|                reliability engine, analyzers, AI module     |
+-------------------------------------------------------------+
| Integration    Adapters: Kafka, Spark, HDFS, Airflow, Docker |
+-------------------------------------------------------------+
| Infrastructure Containers, network, volumes                 |
+-------------------------------------------------------------+
```

5.2 Data path

```
Source (CSV / JSON / REST)
        |
        v
Ingestion producer (adds unique record IDs)
        |
        v
Kafka topic
        |
        v
Spark Structured Streaming job (configuration-driven)
        |
        v
HDFS (processed output)

Metrics from each stage --> Prometheus --> Grafana / Dashboard
```

5.3 Custom contribution versus reused infrastructure

The central contribution of the project is the custom software that decides, generates, controls, measures, and explains. Kafka, Spark, HDFS, Airflow, PostgreSQL, Prometheus, and Grafana are treated as infrastructure that the platform orchestrates.

Developed by the team	Reused infrastructure
Configuration parser and pipeline model	Apache Kafka
Validation engine	Apache Spark
Pipeline generator and execution manager	HDFS
Plugin architecture (registry, factory, strategies)	Apache Airflow
Reliability engine and fault injection	PostgreSQL
Metrics collection, analyzer, report generator	Prometheus and Grafana
REST API, authentication, authorization	Docker and Docker Compose
Web dashboard	
AI analysis module	

6. Technology Stack and Justification

Technology	Role	Justification
Python, FastAPI	Backend, engines	Rich data ecosystem, automatic OpenAPI documentation, type-driven validation
Pydantic	Configuration models	Declarative validation with precise error reporting
SQLAlchemy, Alembic, PostgreSQL	Persistence	Mature ORM, versioned schema migrations, reliable relational storage
React, TypeScript	Dashboard	Component-based UI with static typing
Apache Kafka	Messaging	Durable log with offsets and consumer groups, central to recovery analysis
Apache Spark	Processing	Distributed DataFrame processing and fault-tolerant Structured Streaming
HDFS	Distributed storage	Replicated block storage; ground truth for data verification
Apache Airflow	Workflow orchestration	Scheduling and dependency management of pipeline workflows
Prometheus, Grafana	Monitoring	Standard metrics collection and visualization
Docker, Docker Compose	Environment	Reproducible, isolated local deployment
GitHub Actions	CI	Automated linting, testing, image builds, and security checks
Scikit-learn	AI module	Established implementations of anomaly detection algorithms

7. Development Methodology

7.1 Iterative and incremental development with verification gates

The system is built incrementally in ordered phases. Each phase produces a working, tested increment that is integrated with the previous ones. The team does not defer integration to the end of the project, because the principal technical risk lies in the interaction among independent distributed components.

7.2 Phase gate (definition of done)

No phase is considered complete, and no subsequent phase begins, until all of the following hold:

- All containers build successfully.
- All services start and pass their health checks.
- Automated tests for the phase pass.
- The functionality of the phase is demonstrated against real services.
- Identified defects are fixed and documented.

7.3 Engineering practices

- Version control with Git and GitHub, using feature branches, pull requests, and mandatory peer review.
- Configuration through environment variables; no secrets in the repository; an `.env.example` file with safe development defaults.
- Documentation written together with the code it describes.

- Continuous integration for every change.

7.4 Design principles and patterns

The design follows SOLID, separation of concerns, high cohesion, low coupling, and clean architecture, in which business logic is independent of the web framework, database, and external tools. The principal patterns are Registry and Factory (plugin lookup and creation), Strategy (interchangeable sources, processors, storage backends, and faults), Dependency Injection (testable services), Repository (persistence isolation), and Template Method (a consistent experiment lifecycle across fault types).

8. Work Breakdown and Implementation Phases

The project is divided into fourteen sequential phases. For each phase, the objective, main activities, deliverables, and verification criteria are stated.

Phase 1: Foundation

Objective. Establish the repository, architecture, and containerized base.

Activities

- Create the repository structure and coding standards.
- Create the backend skeleton with a health endpoint, PostgreSQL, Alembic migrations, environment-based configuration, and structured logging.
- Create the Docker network, volumes, and health checks.

Deliverables and verification

- Running backend and database containers; the health endpoint responds; the first migration applies cleanly; baseline tests pass.

Phase 2: Configuration Engine

Objective. Allow pipelines to be defined declaratively and validated rigorously.

Activities

- Define the YAML specification and the Pydantic pipeline model.
- Implement the parser and the validation layers (structural, semantic, and cross-field).
- Implement pipeline create, read, update, delete, and version storage.

Deliverables and verification

- Configuration parser, validator, and pipeline API; unit tests covering valid and invalid configurations, including each error category (missing fields, invalid types, unsupported sources and operations, invalid schema, storage, Kafka, and Spark settings).

Phase 3: Plugin Architecture and Pipeline Execution Manager

Objective. Make the platform extensible and able to coordinate pipeline execution.

Activities

- Define abstract interfaces for sources, processors, and storage; implement the registry and factory.
- Implement the CSV, JSON, and REST API sources; the eight processing operations; and the storage plugins.
- Implement the pipeline generator and the execution manager with enforced lifecycle states.

Deliverables and verification

- Working plugin system demonstrated by adding a new plugin without editing core files; unit tests for the registry, factory, generator, and state transitions.

Phase 4: Kafka Integration

Objective. Provide the messaging layer between ingestion and processing.

Activities

- Implement topic creation, producers, consumers, partition and consumer-group configuration, offset management, and health checks.
- Configure producer delivery settings (acknowledgements, retries, idempotence) and document the resulting delivery semantics.
- Implement readiness checks so dependent services wait for Kafka to be genuinely ready.

Deliverables and verification

- Kafka adapter and ingestion producer; integration tests that publish and consume records, verify offsets, and compute consumer lag.

Phase 5: Spark Integration

Objective. Process data with real Spark jobs inside Docker.

Activities

- Deploy a Spark master and worker.
- Implement a generic, configuration-driven Spark job (batch and Structured Streaming) that builds its transformations from the pipeline model.
- Configure partitioning and checkpoint locations for fault-tolerant processing; integrate with Kafka.

Deliverables and verification

- Spark job and submission logic; integration tests confirming that records flow from Kafka through the configured operations; verification that a restarted job resumes from its checkpoint.

Phase 6: HDFS Integration

Objective. Persist processed data in distributed storage and verify it.

Activities

- Deploy NameNode and DataNode; implement writing, reading, existence checks, health checks, replication settings, and storage metrics.
- Implement output verification used later as ground truth for data-loss measurement.

Deliverables and verification

- HDFS storage plugin; integration tests for write, read, existence, and health; end-to-end run Source to Kafka to Spark to HDFS with matching record counts.

Phase 7: Airflow Integration

Objective. Provide scheduled, dependency-aware workflow execution.

Activities

- Deploy Airflow and implement DAGs for the workflow: validate configuration, prepare source, verify Kafka, run Spark processing, validate output, and collect metrics.

- Keep Airflow tasks thin by delegating logic to the platform's own services.

Deliverables and verification

- Working DAGs; a scheduled run completes and its status is reflected in the platform's run records.

Phase 8: Reliability Engine

Objective. Implement the core ChaosLab capability.

Activities

- Implement the experiment model, validation, and the lifecycle: create, validate, prepare, baseline, inject, monitor, wait for recovery, validate data, collect metrics, analyze, clean up, and report.
- Implement the fault plugins: container stop, container restart, network disruption, CPU stress, memory stress, disk pressure, and service delay.
- Implement the safety controls: target validation, allow-list, project scoping, maximum durations, automatic cleanup, rollback, experiment identifiers, and execution logs.
- Implement the analyzer and the report generator.

Deliverables and verification

- Reliability engine and report generation; safety tests proving that forbidden targets are rejected, maximum duration is enforced, and cleanup runs even when an experiment fails; at least one real experiment per fault type executed against the running pipeline.

Phase 9: Metrics and Monitoring

Objective. Make the system observable.

Activities

- Expose application metrics (records processed, processing latency, errors, recovery time, experiment duration).
- Configure Prometheus scraping and provision Grafana datasources and dashboards automatically.
- Store experiment metric summaries in the database.

Deliverables and verification

- Dashboards that visibly show throughput degradation and recovery during an experiment; metric endpoints covered by tests.

Phase 10: Authentication and Authorization

Objective. Secure the API.

Activities

- Implement password hashing, JWT issuance and validation, and the roles ADMIN, ENGINEER, and VIEWER.
- Enforce permissions at the API layer and record sensitive actions in audit logs.

Deliverables and verification

- Role-based access tests confirming that each role can perform only its permitted operations.

Phase 11: Web Dashboard

Objective. Provide the user interface.

Activities

- Implement the login, dashboard, pipeline management, pipeline details, reliability testing, and reports pages.
- Implement charts for recovery time, throughput, data loss, CPU, memory, and error rate; implement loading, empty, and error states.

Deliverables and verification

- A complete user journey from creating a pipeline to viewing a report through the interface; component tests for critical flows.

Phase 12: Test Consolidation and Continuous Integration

Objective. Consolidate and automate quality assurance.

Activities

- Complete unit, integration, and end-to-end test suites; configure linting and type checking.
- Configure the GitHub Actions workflow: lint, unit tests, integration tests, image builds, security checks, and test reports.

Deliverables and verification

- A passing automated workflow on the main branch; test and coverage reports.

Phase 13: AI Module

Objective. Add optional, explainable analysis of reliability data.

Activities

- Implement anomaly detection using statistical methods and Scikit-learn models such as Isolation Forest and One-Class SVM.
- Implement failure-risk indicators presented as probabilities with stated limitations, root-cause assistance based on correlated metrics, logs, and service states, and evidence-linked recommendations.
- As an extension, evaluate a retrieval-based assistant that answers questions about stored reports and logs.

Deliverables and verification

- AI module integrated behind an interface; the platform operates correctly when the module is disabled; evaluation of detection quality on recorded experiment data.

Phase 14: Documentation and Final Integration

Objective. Deliver a reproducible, demonstrable system.

Activities

- Write the README, architecture documentation with diagrams, plugin and API guides, and the troubleshooting guide.
- Run performance benchmarks and the full demonstration scenario on a clean clone.

Deliverables and verification

- Another developer can clone the repository and run the system by following the documentation alone.

9. Reliability Experiment Methodology

9.1 Experimental design

- **Independent variables:** target service, fault type, and fault duration.
- **Dependent variables:** recovery time, data loss, duplicates, throughput, throughput degradation, availability, error rate, and resource usage.
- **Controlled factors:** identical dataset, identical pipeline configuration, identical baseline period, and identical resource limits across runs.

9.2 Experiment lifecycle

Every experiment follows the same ordered lifecycle: create, validate, prepare, baseline, inject failure, monitor, wait for recovery, validate data, collect metrics, analyze, clean up, and generate report. Cleanup always executes, including after errors or cancellation.

9.3 Fault catalog

Fault	Effect	Reversal
Container stop	Service becomes unavailable	Container restarted
Container restart	Short interruption and return	Health verified
Network disruption	Service alive but unreachable	Network reconnected or rule removed
CPU stress	Resource contention	Stress process terminated
Memory stress	Memory pressure within container limits	Helper terminated
Disk pressure	Bounded, isolated space exhaustion	Temporary file deleted
Service delay	Added network latency	Latency rule removed

9.4 Metric definitions

Metric	Definition
Recovery time	Time from fault injection until the service is healthy and throughput returns to a defined fraction of baseline
Data loss	Expected unique record identifiers minus unique identifiers found in the output
Duplicates	Total output records minus unique output records
Throughput	Records processed per second, reported as average and minimum during the fault
Throughput degradation	Relative drop from baseline during the fault
Latency	Time from ingestion to storage (mean and 95th percentile)
Availability	Healthy samples divided by total samples over the experiment window
Error rate	Failed operations divided by total operations
Resource usage	CPU and memory sampled from container statistics

The precise definition of "recovered" is fixed once and used consistently in the code, the documentation, and the reports.

9.5 Data consistency validation

Each ingested record carries a unique identifier, and optionally a content checksum. After recovery, the identifiers found in HDFS are compared with those that were sent, yielding missing records (loss), repeated records (duplication), and, with checksums, altered records (corruption). When the pipeline filters or aggregates records, the expected output is derived by applying the same logic to the sent records.

9.6 Repeatability and result interpretation

Each experiment configuration is executed multiple times. Results are reported with the mean and the variation across runs, together with the machine specification and resource limits. A transparent composite reliability score may be derived from the measured metrics; its formula and weights are documented, and its sub-scores are always displayed.

9.7 Safety protocol

The engine never executes arbitrary commands on the host. It operates only on containers of the configured project that appear on an explicit allow-list, resolves targets itself rather than from free-form input, uses the container API rather than shell command construction, enforces a maximum duration for every experiment, and guarantees cleanup through error handling, a watchdog timer, and a startup sweep for leftovers. Access to the container runtime is isolated and its security implications are documented.

10. Verification and Validation

Level	Scope	Examples
Unit	Individual components without external services	Configuration parser, validators, plugin registry and factory, generator, experiment state machine, safety checks, metric calculations, report generation
Integration	Platform code with real services	Kafka production and consumption, Spark jobs, PostgreSQL repositories and migrations, HDFS write, read, and health
End to end	Complete workflow	Upload configuration, validate, deploy, ingest, process, store, inject failure, recover, verify data, generate report
Safety	Dangerous operations	Allow-list rejection, maximum-duration enforcement, cleanup after forced errors
Security	Access control	Role-based permission tests, input validation
Frontend	User interface	Component tests and critical user flows

Every requirement of the project is traced to at least one automated test or a documented manual verification procedure. A reliability result is never reported unless the experiment was actually executed against the real services.

11. Evaluation Plan

The project will be evaluated against the following criteria.

Criterion	Method	Success condition
Functional correctness	Automated test suites	All suites pass in continuous integration
Genericity (RQ1)	Run sales, IoT, and network-log datasets using only different YAML files	No core code changes required between datasets

Criterion	Method	Success condition
Extensibility	Add a new plugin without editing core files	Plugin works through the registry alone
Reliability assessment (RQ2)	Execute experiments for each fault type and target, with repetitions	Metrics are produced, stored, and reproducible across runs
Data integrity	Identifier and checksum comparison	Loss and duplicates are measured and reported accurately
Performance	Benchmarks of ingestion and processing throughput, API response time, recovery time, and validation time	Results stored and reported with environment details
Usability	Walkthrough of the full user journey in the dashboard	Journey completed without manual database or container intervention
Reproducibility	Run from a clean clone following the README	System starts and passes the demonstration scenario

12. Risk Analysis and Mitigation

Risk	Impact	Mitigation
High resource consumption of the full stack on a development machine	Slow or unstable environment	Container resource limits, Compose profiles for partial startup, lighter development configuration
Version incompatibilities among Spark, Kafka connector, Scala, and Hadoop	Failing jobs	Pin and document all versions; verify in the phase gate
Services reported as started but not ready	Intermittent failures	Health checks and application-level readiness checks
Fault injection leaves the system in a broken state	Lost experiment validity, damaged environment	Guaranteed cleanup, watchdog, startup sweep, maximum durations
Misuse of container runtime access	Host-level exposure	Isolation, restricted API surface, allow-list, project scoping, documentation of risks
Single-node limitations reduce realism of replication behavior	Limited conclusions	State limitations explicitly; design experiments around what the environment can demonstrate
Ambiguous metric definitions	Meaningless comparisons	Single documented definition per metric, used everywhere
Insufficient historical data for AI models	Weak detection quality	Treat AI as supplementary; report limitations and data requirements; use statistical baselines
Integration problems discovered late	Schedule disruption	Phase gates with integration verification at each phase
Unequal progress among work streams	Bottlenecks	Clear interfaces between components, peer review, regular progress review sessions

13. Team Organization

Responsibilities are divided into work streams. Each stream has a lead and a reviewer from a different stream so that every component is understood by more than one team member.

Work stream	Main responsibilities
Backend and API	FastAPI application, database design and migrations, authentication, audit logging
Pipeline and Big Data engine	Configuration engine, plugins, generator, execution manager, Kafka, Spark, HDFS, Airflow integrations
Reliability and DevOps	Docker environment, reliability engine, fault plugins, safety controls, monitoring, CI
Frontend and AI	Dashboard, visualizations, AI module, report presentation
Quality and documentation (shared)	Test strategy, benchmarks, documentation, demonstration preparation

Coordination relies on a shared task board, regular review sessions, written interface definitions, and pull-request reviews.

14. Quality Assurance and Standards

- Python: Black for formatting, Ruff for linting, MyPy for type checking where practical, Pytest for testing.
- Frontend: ESLint, Prettier, and strict TypeScript.
- Clear naming, docstrings for important components, and minimal unnecessary comments.
- Structured JSON logging with timestamp, service, pipeline identifier, experiment identifier, level, message, and error details.
- Consistent error handling: errors are logged, associated with identifiers, and returned as meaningful responses without crashing the platform.

15. Ethical, Legal, and Safety Considerations

- **Experiment safety.** Fault injection is restricted to the project's own containers and is time-limited and reversible.
- **Data.** All datasets are synthetic or publicly available; no personal or confidential data is processed.
- **Secrets.** Credentials are supplied through environment variables and are never committed to version control.
- **Licensing and attribution.** Third-party software is used under its open-source licenses and is credited in the documentation.
- **Honest reporting.** Limitations of the environment, the experiments, and the AI module are stated explicitly, and no claim of success is made without executed evidence.

16. Project Deliverables

Deliverable	Description
Source code	Backend, frontend, engines, plugins, Spark jobs, and Airflow DAGs
Infrastructure	Dockerfiles, Docker Compose configuration, Prometheus and Grafana configuration
Database	Schema and migrations
Datasets	Sales, IoT, and network-log samples with a data generator
Experiments	Example experiment definitions
Tests	Unit, integration, end-to-end, and safety tests
CI workflow	GitHub Actions pipeline
AI module	Anomaly detection, risk indication, recommendations
Documentation	README, architecture and data-flow documents, diagrams, plugin guide, API documentation, troubleshooting guide
Evaluation results	Benchmark results and reliability reports
Demonstration	Prepared end-to-end demonstration scenario

17. Acceptance Criteria

The project is considered complete when all of the following are demonstrated:

- A user can create a YAML pipeline configuration, and it is validated with useful error messages.
- The system deploys the required resources, data enters Kafka, Spark processes it, and results are stored in HDFS.
- Metrics are collected and visualized.
- A failure can be injected safely, the system recovers where the architecture supports recovery, and recovery time and data consistency are measured.
- A reliability report is generated, stored, and viewable in the dashboard.
- The optional AI module analyzes historical metrics, and the platform works without it.
- Automated tests exist and pass.
- Another developer can clone the repository and run the system by following the documentation.

18. Limitations and Future Work

18.1 Limitations

The evaluation is performed on a single development machine with container-level faults, so some phenomena of real multi-node clusters, such as multi-broker replication behavior and genuine hardware failure, can only be approximated. The quality of the AI module is bounded by the amount of experiment data that can be generated in a student environment.

18.2 Future work

- Deployment on multi-node clusters and Kubernetes.
- Additional fault types, including partial and gradual failures.
- Additional connectors and storage backends through new plugins.

- Scheduled and continuous reliability testing integrated with deployment workflows.
- A retrieval-based reliability assistant and more advanced predictive models.

19. Conclusion

ChaosFlow addresses two practical problems of distributed data engineering: the repeated effort of building pipelines and the lack of systematic reliability verification. The plan combines an incremental, gate-controlled development method with a rigorous experimental methodology and an explicit evaluation framework. Following this plan, the team will deliver a modular, testable, observable, and reproducible platform that demonstrates both software-engineering depth and an evidence-based approach to resilience.

20. Selected References

- Armbrust, M., et al. (2018). Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark. *Proceedings of the ACM SIGMOD International Conference on Management of Data.*
- Basiri, A., et al. (2016). Chaos Engineering. *IEEE Software, 33(3).*
- Basiri, A., et al. (2019). Automating Chaos Experiments in Production. *IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP).*
- Beyer, B., Jones, C., Petoff, J., and Murphy, N. R. (2016). *Site Reliability Engineering.* O'Reilly Media.
- Kreps, J., Narkhede, N., and Rao, J. (2011). Kafka: A Distributed Messaging System for Log Processing. *Proceedings of the NetDB Workshop.*
- Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation Forest. *IEEE International Conference on Data Mining (ICDM).*
- Shvachko, K., Kuang, H., Radia, S., and Chansler, R. (2010). The Hadoop Distributed File System. *IEEE Symposium on Mass Storage Systems and Technologies (MSST).*
- Zaharia, M., et al. (2012). Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. *USENIX Symposium on Networked Systems Design and Implementation (NSDI).*